

Problem Statement and Goals

Tandem

Team 11, Tandem
Justin Kwinecki
William Pol
Zayed Umer
Musab Mondal
Manav Patel

Table 1 records the changes to this document.

Table 1: Revision History

Date	Developer(s)	Change
Sept 22, 2026	Justin Kwinecki	Initial problem statement and goals
Sept 27, 2026	Zayed Umer	Revise problem statement and goals; see disclosure appendix

1 Problem Statement

Tandem addresses the cost of repeatedly processing the same background information for several co-operating software agents. This section defines that problem, the people affected, and the intended operating environment.

1.1 Problem

A large language model (LLM) generates text from an input prompt. An agent is a software component that uses a model to carry out a task. Several agents may work on different tasks using the same background information: for example, one agent summarizes a document while another extracts its key dates. Each agent submits the document followed by its own question. The common beginning of these prompts is their shared prefix.

Before producing an answer, the model processes the prompt in a phase called prefill. If several requests independently process the same long prefix, they repeat work. On a server with limited computing capacity, this repeated work can delay responses and leave less capacity for other requests. Application developers and server operators need to serve these requests efficiently while preserving each agent’s intended result.

Existing serving software can reuse previously processed prefixes [1]. Tandem’s open question is how much repeated work remains when related requests arrive together, or arrive while their common prefix is still being processed. The project will measure this behaviour before claiming an improvement. The problem is to reduce avoidable prompt processing and response delays in these workloads while keeping each request independent. Particular algorithms and engine changes belong in the design documentation.

1.2 Scope

The core scope is text generation for groups of concurrent requests that use the same model and share an identical prefix, followed by different agent instructions. Requests may arrive together or a short time apart. Requests with no shared prefix are also included as a compatibility check. Evaluation will compare the same workloads, model, hardware, and generation settings with and without Tandem’s changes, including the serving engine’s existing prefix reuse features in the baseline.

The project uses existing models; it does not train a new model or improve the quality of an agent’s reasoning. Agent orchestration, document retrieval, and application user interfaces are outside the core scope. Semantically similar but differently worded context is not considered an identical prefix. General prompt trees with several branching levels and operation across multiple servers are beyond the initial scope.

The proof of concept must establish that the selected baseline repeats work and that the proposed processing preserves model results. If the expected gap is absent or cannot be reduced reliably, the fallback is to measure and report repeated prompt processing in existing serving systems. Such a change would require revising these goals with the teaching assistant (TA) and instructor. The [Development Plan](#) describes project execution and the proof of concept plan.

1.3 Inputs and Outputs

The inputs are:

- Text-generation requests, each with an identifier, an ordered prompt containing its context and instructions, and generation settings such as the maximum response length and sampling controls.
- A selected, already trained model and its text encoding rules, which determine the tokens (units of text) the model receives.
- For evaluation, a workload specifying the requests and their arrival times, together with the serving configuration.

The outputs are:

- A generated response associated with each accepted request, and a completion status or an explicit failure indication.
- Evaluation results showing response timing, completed requests per second, prompt tokens actually processed, and agreement with the baseline’s model outputs. Each result identifies the workload and configuration needed to reproduce it.

Each response depends only on its own prompt and generation settings; another agent’s private instructions must not influence it. Internal cached state is an implementation detail, rather than a user-facing output.

1.4 Stakeholders

- **Application developers, including independent developers:** submit related requests and need predictable results and shorter waits without having to redesign their agent tasks.
- **Serving-engine developers and maintainers:** integrate and maintain changes, and need clear interfaces, correctness evidence, and compatibility with ordinary requests.

- **Infrastructure operators and deploying organizations:** allocate computing resources and need evidence that the same hardware can serve shared-context workloads more efficiently. This includes operators of self-hosted systems and cloud infrastructure.
- **Application end users:** benefit from faster responses and depend on their requests remaining separate from other agents’ work.
- **The capstone team, TA, and instructor:** develop or evaluate the project and need a feasible scope, reproducible results, and evidence that the implementation meets its stated goals.

1.5 Environment

The intended users are developers and operators running an LLM serving system on a local or remote Linux server with a supported graphics processing unit (GPU) and enough memory for the selected model and requests. Applications submit requests through a programmatic interface; end users interact with those applications. The initial evaluation targets one server and one supported model configuration at a time.

The operator supplies the model, compatible inference software, and access to the server. Performance depends on the model, available memory, prompt lengths, and request arrival pattern, so reported results must identify those conditions. Developer tools and the team’s laboratory hardware are described in the Development Plan.

2 Goals

The following goals describe the intended benefits. The Software Requirements Specification and Verification and Validation Plan will refine them into numerical acceptance criteria and test procedures before final evaluation. Performance comparisons use the matched baseline from Section 1.2, repeated runs, and reported measurement variation; a change within that variation is not evidence of an improvement.

- G1. **Reduce repeated prompt processing.** Process fewer prompt tokens for shared-prefix workloads than the baseline, measured from actual execution rather than the total number of submitted tokens. Compare simultaneous and staggered arrivals, with both previously processed and unseen prefixes.
- G2. **Reduce response delay.** Lower the median and 95th-percentile time to first token for the target workloads. Time to first token is the elapsed time from submitting a request until its first generated token is received, including any wait before processing begins.
- G3. **Increase useful serving capacity.** Complete more requests per second on the same hardware for shared-prefix workloads with matched response lengths, while reporting the accompanying response delays.
- G4. **Preserve model behaviour and request independence.** Match the reference model’s output scores within a documented numerical tolerance under controlled settings, and pass tests showing that changing one agent’s private instructions does not change another agent’s output scores. Include multi-token generation in the correctness evaluation.

- G5. **Preserve ordinary serving.** Continue to handle individual requests and requests without shared context, passing the same functional checks as the baseline. Measure their response delay and throughput to identify overhead introduced by Tandem.

These are project targets, not achieved results. The evaluation will report workloads where the approach provides no benefit or performs worse, as well as those where it helps.

3 Stretch Goals

After the core goals are met, the team may pursue the following extensions:

- **Support deeper branching.** Extend correctness and performance evaluation to prompts with several shared branching points, including branches introduced during an ongoing interaction.
- **Broaden demonstrated applicability.** Repeat the evaluation on additional models or GPU configurations and identify which benefits persist across them.
- **Reduce memory demand.** Use less peak GPU memory than the matched baseline for shared-context workloads and measure whether this permits more concurrent requests within the same memory limit.

4 Custom Documents (CDs)

The planned custom documents match the choices in the team’s Development Plan draft. Their topics and scope remain subject to instructor approval.

- **Fall: Machine Learning Model Report.** Explain the existing model selected for the proof of concept, the evaluation prompts and tasks, and the checks used to compare model behaviour under different processing arrangements. Explain why training and training-data splits do not apply to this inference project, and discuss relevant model and evaluation-data limitations.
- **Winter: Performance Report.** Present controlled experiments on repeated processing, response delay, serving capacity, and resource use. Record hardware, software versions, workloads, repeated trials, measurement variation, and failure cases. This report will provide the detailed performance analysis; the Verification and Validation Report will summarize whether the requirements were met and refer to it.

References

- [1] vLLM contributors. Automatic prefix caching. https://docs.vllm.ai/en/latest/features/automatic_prefix_caching/. Accessed September 27, 2026.

Appendix — Generative AI Use Disclosure

OpenAI Codex assisted with this revision by reading the repository’s existing problem statement, Development Plan draft, problem-statement and writing checklists, and the supplied course outline. It drafted and edited the problem framing, scope, inputs and outputs, stakeholders, environment, goals, and custom-document descriptions. It also checked the cited prefix-caching documentation and prepared the LaTeX source and PDF.

The initial problem statement and goals were written by Justin Kwinecki. This revision also considered the stakeholder and input/output additions in his open problem-statement pull request. Codex did not run performance experiments or establish the proposed optimization’s novelty. The goals are proposals for team review. Zayed Umer supplied the reflection notes about collaborative brainstorming, the difficulty of finding a novel idea, and the motivation to improve memory use even modestly. Codex edited those notes into the reflection below and connected them to this document’s scope. The team must review the document and remains responsible for its accuracy and scope.

Appendix — Reflection

These responses draw on the team’s experience as described by Zayed Umer.

1. What went well while writing this deliverable?

We agreed early that we wanted to work on inference, so brainstorming was collaborative. Having a shared area of interest let us build on each other’s ideas and concentrate on choosing a useful problem within that area. This gave us a common starting point for the problem statement.

2. What pain points did you experience during this deliverable, and how did you resolve them?

Finding a novel idea was harder than agreeing on the general topic. The amount of inference research made it difficult to judge whether an idea addressed an open problem or repeated existing work. We have not fully resolved that uncertainty. The plan is to compare against existing work and measure the selected serving engine before claiming that Tandem adds something new. Making that uncertainty explicit gives the proof of concept a concrete question to answer.

3. How did you and your team adjust the scope of your goals to ensure they are suitable for a Capstone project?

Our starting motivation was to make inference more resource-efficient, especially in memory use, even if the improvement was modest. That motivation needs a narrower target than improving inference in general. This document focuses on requests that share the same context, using existing models and a matched baseline so that any improvement can be measured. Correctness and reduced repeated processing come first; additional memory savings and broader model support are stretch goals. This leaves substantial work in serving-system design and evaluation while avoiding a commitment to build a new model or a complete inference platform. A small, reproducible gain would be more useful than a large performance claim that we could not explain or verify.